



■ Guia Didático dos Scripts Python

Energix Global

Professor de Ciência de Dados | MIT Style

Para estudantes iniciantes em Python e Machine Learning

Arquivo	Tópico Principal
	Pacotes Python
process_data.py	ETL Pipeline + Pandas + argparse
	K-Means Clustering (scikit-learn)
forecast.py	Regressão Polinomial + Métricas ML
	Análise Exploratória de Dados

Renan Pinheiro · 2026 · Version 1.0.0

Guia Didático dos Scripts Python Energix Global

Professor de Ciência de Dados | MIT Style

Como usar este guia: Leia linearmente, do início ao fim. Cada conceito é apresentado antes de ser utilizado. Os blocos de código são mostrados exatamente como estão nos arquivos, seguidos de explicações linha a linha.

Viso Geral dos 4 Arquivos

scripts/	
■■■■ __init__.py	→ Marca o diretório como um pacote Python
■■■■ process_data.py	→ O "maestro" – orquestra tudo e gera o data.json
■■■■ clustering.py	→ Machine Learning: agrupa países por perfil energético
■■■■ forecast.py	→ Machine Learning: prevê o futuro com matemática
■■■■ eda.py	→ Análise exploratória: entende os dados antes de tudo

Analogia: Imagine que você é um chef de cozinha.

- `eda.py` é o momento em que você **prova e inspeciona os ingredientes**
- `clustering.py` é quando você **separa os ingredientes por categoria**
- `forecast.py` é quando você **prevê quantos pratos venderá amanhã**
- `process_data.py` é a **receita completa** que une tudo

ARQUIVO 1: `\_\_init\_\_.py`

```
# Energix Global – scripts package
```

O que é isso?

Este arquivo está **propositalmente vazio** (só tem um comentário). Sua única função é existir. Em Python, quando você coloca um arquivo chamado `__init__.py` dentro de uma pasta, você está dizendo ao Python:

*“Ei Python, esta pasta não é só uma pasta qualquer — ela é um **pacote**, um conjunto organizado de módulos que podem ser importados.”*

Sem este arquivo, quando `process_data.py` tenta fazer:

```
from scripts.clustering import run_clustering
```

Python responderia: *“Nunca ouvi falar de nenhum pacote chamado `scripts`.”*

Com este arquivo, Python entende que `scripts/` é um pacote e consegue localizar `clustering.py` dentro dele.

ARQUIVO 2: `process_data.py`

O Pipeline Principal

Este é o arquivo mais importante. Ele é o **maestro da orquestra** — não toca nenhum instrumento, mas coordena todos os outros para criar a música final (o `data.json`).

■ BLOCO 1 — Imports (linhas 27–41)

```
import argparse
import json
import logging
import os
import sys
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

warnings.filterwarnings("ignore")
```

O que são imports?

Imports são como **pegar ferramentas da caixa de ferramentas** antes de começar um trabalho. Python vem com algumas ferramentas nativas e outras precisam ser instaladas.

Import	Tipo	Para que serve
<code>argparse</code>	Nativo Python	Lê argumentos passados no terminal (ex: <code>--dataset arquivo.csv</code>)
<code>json</code>	Nativo Python	Lê e escreve arquivos <code>.json</code>
<code>logging</code>	Nativo Python	Imprime mensagens de acompanhamento no terminal
<code>os</code>	Nativo Python	Interage com o sistema operacional (arquivos, pastas)
<code>sys</code>	Nativo Python	Controla a execução do programa (ex: encerrar com erro)
<code>warnings</code>	Nativo Python	Controla avisos exibidos pelo Python
<code>pathlib.Path</code>	Nativo Python	Manipula caminhos de arquivos de forma elegante
<code>numpy (np)</code>	Instalado	Matemática com arrays e matrizes — o "Excel do Python"
<code>pandas (pd)</code>	Instalado	Tabelas de dados — o "Excel avançado do Python"
<code>sklearn.cluster.KMeans</code>	Instalado	Algoritmo de agrupamento K-Means
<code>sklearn.linear_model.LinearRegression</code>	Instalado	Regressão linear
<code>sklearn.preprocessing.PolynomialFeatures</code>	Instalado	Cria features polinomiais
<code>sklearn.preprocessing.StandardScaler</code>	Instalado	Normaliza os dados

Por que `import numpy as np` ?

O `as np` é um apelido. Em vez de escrever `numpy.array([1,2,3])` toda vez, você escreve só `np.array([1,2,3])` . É pura conveniência — `np` para NumPy e `pd` para pandas são convenções universais em Ciência de Dados.

```
warnings.filterwarnings("ignore")
```

Suprime avisos técnicos que o scikit-learn às vezes exibe durante o treinamento de modelos. Em produção, é uma boa prática suprimir avisos que você já conhece e não representam erros reais.

■ BLOCO 2 — Configuração de Logging (linhas 43–49)

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-8s %(message)s",
    datefmt="%H:%M:%S",
)
log = logging.getLogger("energix")
```

O que é logging?

Imagine que você está pilotando um avião. O **log de voo** registra tudo que acontece: decolagem, altitude, combustível, pouso. O **logging** do Python faz a mesma coisa para o seu programa.

Por que não usar `print()` simplesmente?

`print()` funciona, mas **logging** é muito mais profissional:

- Adiciona **horário automático** a cada mensagem
- Tem **níveis de gravidade**: DEBUG < INFO < WARNING < ERROR < CRITICAL
- Pode ser **ligado/desligado** facilmente
- Em produção, você pode salvar os logs em arquivo

Decodificando o `format` :

"%(asctime)s"	"%(levelname)-8s"	"%(message)s"
↑	↑	↑
Horário	Nível (INFO,	A mensagem
(09:20:48)	WARNING...)	que você escreveu

O `-8s` no `%(levelname)-8s` significa: "reserve 8 caracteres para o nível, alinhando à esquerda" — deixa a saída bonitinha no terminal.

Resultado no terminal:

```
09:20:48 INFO      Carregando dataset: /home/.../dataset_renewable.csv
09:20:48 INFO      Dataset carregado — shape: (7636, 33)
```

`log = logging.getLogger("energix")` cria um logger com o nome "energix". Assim, se você tiver múltiplos módulos, cada um com seu próprio nome (`energix.clustering` , `energix.forecast`), fica fácil identificar **qual módulo** gerou cada mensagem.

■ BLOCO 3 — Caminhos Padrão (linhas 51–54)

```
ROOT = Path(__file__).resolve().parent.parent
DEFAULT_DATASET = ROOT / "dataset" / "dataset_renewable.csv"
DEFAULT_OUTPUT = ROOT / "dashboard" / "data"
```

`Path(__file__).resolve().parent.parent`

Vamos desmontar isso peça por peça:

Expressão	O que retorna	Exemplo
<code>__file__</code>	Caminho do arquivo atual	<code>/home/user/energixglobal/scripts/processdata.py</code>
<code>Path(...)</code>	Converte para objeto Path	Idem, mas agora manipulável
<code>.resolve()</code>	Caminho absoluto completo	Remove <code>..</code> e links simbólicos
<code>.parent</code>	Pasta que contém o arquivo	<code>/home/user/energix_global/scripts/</code>
<code>.parent.parent</code>	Pasta acima da pasta acima	<code>/home/user/energix_global/</code>

Então `ROOT` aponta para a **raiz do projeto**, independente de onde o script for executado.

O operador `/` com Path

```
ROOT / "dataset" / "dataset_renewable.csv"
```

O `/` aqui **não é divisão** — é o operador de concatenação de caminhos do `pathlib`. É equivalente a:

```
os.path.join(ROOT, "dataset", "dataset_renewable.csv")
```

Mas muito mais legível. E funciona em qualquer sistema operacional (Windows usa `\`, Linux/Mac usa `/`).

■ BLOCO 4 — Mapeamento de Continentes (linhas 57–108)

```
CONTINENT_MAP: dict[str, list[str]] = {
    "África": [
        "Algeria", "Angola", "Benin", ...
    ],
    "Ásia": [...],
    ...
}

COUNTRY_TO_CONTINENT: dict[str, str] = {
    country: continent
    for continent, countries in CONTINENT_MAP.items()
    for country in countries
}
```

O que é um dicionário (`dict`) em Python?

Um dicionário é como uma **agenda telefônica**: você procura pelo nome (chave) e encontra o número (valor).

```
agenda = {
    "João": "99999-1111",
    "Maria": "88888-2222",
}
agenda["João"]  # retorna "99999-1111"
```

`CONTINENT_MAP` mapeia cada continente para uma lista de países:

```
{
    "África": ["Algeria", "Angola", ...], # chave → lista de valores
    "Ásia":  ["China", "India", ...],
}
```

`COUNTRYTOCONTINENT` inverte essa lógica — dado um país, retorna o continente. Isso é feito com uma **dict comprehension** (compreensão de dicionário):

```
COUNTRY_TO_CONTINENT = {
    country: continent
    for continent, countries in CONTINENT_MAP.items()
    for country in countries
}
```

O que é uma comprehension?

É uma forma compacta de criar listas ou dicionários. O código acima é equivalente a:

```
COUNTRY_TO_CONTINENT = {}  
for continent, countries in CONTINENT_MAP.items():  
    for country in countries:  
        COUNTRY_TO_CONTINENT[country] = continent
```

O resultado é:

```
{  
    "Algeria": "África",  
    "Angola": "África",  
    "China": "Ásia",  
    ...  
}
```

Type hints: `dict[str, list[str]]`

O `: dict[str, list[str]]` após o nome da variável é uma **anotação de tipo** (type hint). Ela não muda o funcionamento do código, mas documenta que `CONTINENT_MAP` é um dicionário cujas chaves são strings e cujos valores são listas de strings. É como um contrato que ajuda outros desenvolvedores (e você mesmo no futuro) a entender o código.

■ BLOCO 5 — Função `load_dataset` (linhas 124–146)


```
def load_dataset(path: Path) -> pd.DataFrame:
    """Carrega e valida o dataset CSV."""
    log.info("Carregando dataset: %s", path)

    if not path.exists():
        log.error("Arquivo não encontrado: %s", path)
        sys.exit(1)

    df = pd.read_csv(path)
    log.info("Dataset carregado – shape: %s", df.shape)

    required_cols = [
        "country", "year", "iso_code",
        "renewables_share_elec", "solar_share_elec",
        "wind_share_elec", "hydro_share_elec",
        "nuclear_share_elec", "fossil_share_elec",
    ]
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        log.error("Colunas ausentes no dataset: %s", missing)
        sys.exit(1)

    return df
```

Anatomia de uma função Python:

```
def nome_da_funcao(parametro: Tipo) -> TipoDeRetorno:
    """Docstring: explica o que a função faz."""
    # corpo da função
    return resultado
```

- `def` — palavra reservada que define uma função
- `path: Path` — parâmetro com type hint (o caminho do arquivo)
- `-> pd.DataFrame` — indica que a função retorna um DataFrame
- A string entre `"""` é a **docstring** — documentação embutida

`pd.read_csv(path)`

Esta é a operação mais fundamental em Ciência de Dados com pandas. Ela lê um arquivo CSV (planilha de texto separado por vírgulas) e cria um **DataFrame** — uma tabela em memória com linhas e colunas.

Arquivo CSV:		DataFrame:
country,year,renewables...	→	country year renewables
Brazil,2025,86.6...		Brazil 2025 86.6
China,2025,37.0...		China 2025 37.0

`df.shape` retorna uma tupla (`linhas, colunas`) . Ex: `(7636, 33)` = 7636 linhas e 33 colunas.

List comprehension para validação:

```
missing = [c for c in required_cols if c not in df.columns]
```

Cria uma lista com todas as colunas de `required_cols` que **não estão** no DataFrame. Se a lista não for vazia, o dataset está incompleto.

```
sys.exit(1)
```

Encerra o programa imediatamente com código de erro `1` (qualquer número diferente de 0 indica erro no Unix/Linux). É a forma padrão de dizer "algo deu errado, não posso continuar".

■ BLOCO 6 — Função `filter_real_countries` (linhas 153–167)

```
def filter_real_countries(df: pd.DataFrame) -> pd.DataFrame:
    mask = (
        df["iso_code"].notna()
        & (df["iso_code"].str.len() == 3)
        & (df["year"] >= 2000)
        & (df["year"] <= 2025)
    )
    df_real = df[mask].copy()
    return df_real
```

O conceito de máscara booleana (boolean mask)

Esta é uma das operações mais importantes do pandas. Uma **máscara** é uma coluna de valores `True` / `False` que indica quais linhas manter.

```
df["year"] >= 2000 retorna:
linha 0: True    (ano = 2024)
linha 1: False   (ano = 1999)
linha 2: True    (ano = 2015)
...

df[mask] filtra só as linhas True.
```

Por que usar `iso_code.str.len() == 3` ?

O dataset contém não só países, mas também regiões agregadas como "ASEAN (Ember)", "World", "Europe", etc. Esses grupos **não têm** código ISO de 3 letras (como **BRA** para Brasil, **CHN** para China). Filtrar por comprimento 3 garante que ficamos apenas com países reais.

O operador `&&` vs `and`

Em pandas, você usa `&&` (e comercial) em vez de `and` para combinar condições. Isso porque `&&` opera elemento a elemento em toda a coluna, enquanto `and` só funciona com valores únicos.

`.copy()`

`df[mask].copy()` cria uma **cópia independente** do DataFrame filtrado. Sem o `.copy()`, alterações no `df_real` poderiam afetar o `df` original — o que causaria o famoso aviso `SettingWithCopyWarning` do pandas. Sempre use `.copy()` quando for modificar um subconjunto.

■ BLOCO 7 — Função `build_summary` (linhas 181–210)

```
def build_summary(world: pd.DataFrame) -> dict:

    def _w(year: int, col: str) -> float:
        row = world[world["year"] == year]
        return float(row[col].values[0]) if len(row) else 0.0

    ren_2000 = _w(2000, "renewables_share_elec")
    ren_2025 = _w(2025, "renewables_share_elec")
    growth_pp = ren_2025 - ren_2000
    growth_pct = round((growth_pp / ren_2000) * 100, 1) if ren_2000 else 0.0
```

Funções dentro de funções (nested functions)

`w` é uma função definida **dentro** de `buildsummary`. Ela só existe dentro dessa função. O prefixo `_` (underscore) é uma convenção Python que indica: *"esta função é de uso interno, não use de fora"*.

Por que criar essa função auxiliar? Para evitar repetição de código. Sem ela, precisaríamos escrever a filtragem por ano e extração de valor 6 vezes (para cada métrica). Com ela, escrevemos uma vez e chamamos 6 vezes.

`row[col].values[0]`

- `row[col]` → seleciona a coluna `col` do DataFrame filtrado
- `.values` → converte de pandas Series para array NumPy

- `[0]` → pega o primeiro (e único) elemento

`if len(row) else 0.0` — operador ternário

É uma forma compacta de escrever:

```
if len(row) > 0:
    return float(row[col].values[0])
else:
    return 0.0
```

Pontos percentuais (pp) vs porcentagem (%)

```
growth_pp = ren_2025 - ren_2000          # 33.76 - 18.72 = 15.04 pp
growth_pct = (growth_pp / ren_2000) * 100 # 15.04 / 18.72 * 100 = 80.3%
```

Atenção: isso é uma distinção **muito importante** em análise de dados:

- **Pontos percentuais (pp)**: diferença absoluta entre dois percentuais. $33.76\% - 18.72\% = 15.04$ pp
- **Variação percentual (%)**: crescimento relativo. $(15.04 / 18.72) \times 100 = 80.3\%$

■ BLOCO 8 — Funções de Séries Temporais

```
def build_global_trend(world: pd.DataFrame) -> list[dict]:
    cols = ["year", "renewables_share_elec", "solar_share_elec", ...]
    trend = []
    for _, row in world[cols].iterrows():
        trend.append({k: (int(v) if k == "year" else round(float(v), 3))
                      for k, v in row.items()})
    return trend
```

`iterrows()`

Percorre um DataFrame linha por linha. Retorna pares (índice, linha) . O `_` é a convenção para "não me importo com o índice, só quero a linha".

Dict comprehension com condicional:

```
{k: (int(v) if k == "year" else round(float(v), 3)) for k, v in row.items()}
```

Para cada par (chave, valor) da linha:

- Se a chave for "year", converte para inteiro
- Caso contrário, arredonda para 3 casas decimais

Por que converter para float() e depois round() ?

Os dados vêm do pandas como tipos especiais (`numpy.float64`). Convertamos para `float` padrão do Python para garantir compatibilidade com o `json.dump()` .

■ BLOCO 9 — Função `buildannualgrowth`

```
def build_annual_growth(world: pd.DataFrame) -> list[dict]:
    ws = world.sort_values("year").copy()
    ws["yoy"] = ws["renewables_share_elec"].diff()
    return [...]
```

`.diff()` — Diferença entre linhas consecutivas

Esta é uma operação muito usada em séries temporais. Ela calcula a diferença entre cada valor e o valor da linha anterior:

Ano	Renováveis	.diff() (YoY)	
2000	18.72	NaN	← não há linha anterior
2001	18.04	-0.67	← 18.04 - 18.72
2002	17.91	-0.13	← 17.91 - 18.04
2025	33.76	+1.83	← 33.76 - 31.94

YoY = *Year over Year* = variação ano a ano.

■ BLOCO 10 — Função `build_ranking`

```
def build_ranking(df: pd.DataFrame, df_clust: pd.DataFrame) -> list[dict]:
    ...
    df_rank["continent"] = df_rank["country"].map(COUNTRY_TO_CONTINENT).fillna("Outros")
    df_rank = df_rank.merge(
        df_clust[["country", "cluster_name"]], on="country", how="left"
    )
    df_rank = df_rank.sort_values("renewables_share_elec", ascending=False).reset_index(drop=True)
    df_rank["rank"] = df_rank.index + 1
```

`.map()` com dicionário

`df["country"].map(COUNTRY7OCONTINENT)` substitui cada valor da coluna `country` pelo valor correspondente no dicionário. É como um `VLOOKUP` do Excel.

```
"Brazil" → "América do Sul"
"China"  → "Ásia"
"France" → "Europa"
```

`.fillna("Outros")` — Preenche valores ausentes

Países que não estão no nosso mapeamento (ex: territórios ou dependências) teriam `NaN` (Not a Number — o "vazio" do pandas). `.fillna("Outros")` substitui esses vazios pela string `"Outros"`.

`.merge()` — Junção de tabelas (JOIN)

```
df_rank.merge(df_clust[["country", "cluster_name"]], on="country", how="left")
```

O `merge` é o equivalente ao `JOIN` do SQL. Estamos adicionando a coluna `cluster_name` ao ranking:

- `on="country"` → coluna de junção (chave)
- `how="left"` → mantém **todos** os países do `df_rank`, mesmo que não tenham cluster

df_rank:			df_clust:		Resultado:		
country	renew		country	cluster	country	renew	cluster
Norway	98.6	■■■■■	Norway	Wind Ch.	→ Norway	98.6	Wind Ch.
Brazil	86.6	■■■■■	Brazil	Hydro L.	Brazil	86.6	Hydro L.

`.sort_values(ascending=False)`

Ordena do maior para o menor (para o ranking fazer sentido — 1º lugar = maior participação renovável).

`.reset_index(drop=True)`

Após filtrar e ordenar, os índices das linhas ficam bagunçados (ex: 0, 5, 12, 3...). Este método redefine os índices de 0 a N-1 em ordem sequencial. `drop=True` descarta o índice antigo.

```
df["rank"] = df.index + 1
```

Como o índice agora começa em 0 e rankings começam em 1, somamos 1.

■ BLOCO 11 — Pipeline Principal `run_pipeline`

```
def run_pipeline(dataset_path: Path, output_dir: Path) -> None:
    df = load_dataset(dataset_path)
    df_real = filter_real_countries(df)
    world = get_world(df)
    summary = build_summary(world)
    ...

    output_dir.mkdir(parents=True, exist_ok=True)
    output_path = output_dir / "data.json"
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)
```

`output_dir.mkdir(parents=True, exist_ok=True)`

Cria a pasta de saída se ela não existir:

- `parents=True` → cria também as pastas intermediárias (como `mkdir -p` no Linux)
- `exist_ok=True` → não dá erro se a pasta já existir

`with open(...) as f:`

O `with` é um **gerenciador de contexto**. Ele garante que o arquivo será fechado automaticamente ao sair do bloco, mesmo que ocorra um erro. É equivalente a:

```
f = open(output_path, "w", encoding="utf-8")
try:
    json.dump(data, f, ...)
finally:
    f.close()
```

- `"w"` = modo de escrita (write) — cria ou sobrescreve o arquivo
- `encoding="utf-8"` = codificação de caracteres (garante que acentos funcionem)

`json.dump(data, f, ensure_ascii=False, indent=2)`

Serializa o dicionário Python para JSON:

- `ensure_ascii=False` → permite caracteres especiais (ã, ç, é) no JSON
- `indent=2` → formata o JSON com indentação de 2 espaços (legível por humanos)

■ BLOCO 12 — `argparse` e Entry Point

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(...)
    parser.add_argument("--dataset", type=Path, default=DEFAULT_DATASET, ...)
    parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT, ...)
    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    run_pipeline(args.dataset, args.output)
```

`argparse` — argumentos de linha de comando

Permite ao usuário passar parâmetros ao executar o script:

```
python -m scripts.process_data --dataset meus_dados.csv --output resultados/
```

`--dataset` e `--output` são argumentos **opcionais** com valores padrão. Se o usuário não passar nenhum, usam `DEFAULTDATASET` e `DEFAULTOUTPUT`.

```
if __name__ == "__main__":
```

Esta é uma das construções mais icônicas do Python. Ela significa: *"execute o código abaixo APENAS se este arquivo for chamado diretamente, não se for importado por outro script"*.

```
python process_data.py          → __name__ == "__main__" → executa o pipeline
from scripts import process_data → __name__ == "scripts.process_data" → NÃO executa
```

Isso permite que o arquivo seja tanto **executável** (script) quanto **importável** (módulo).

ARQUIVO 3: `clustering.py`

Machine Learning com K-Means

■ Conceito Fundamental: O que é K-Means?

Antes de ver o código, entenda o algoritmo.

Problema: Temos 191 países com dados de energia. Queremos agrupá-los em grupos de países **similares**, sem saber de antemão quais são os grupos. Isso se chama **aprendizado não supervisionado** — não há resposta certa, o algoritmo descobre os padrões sozinho.

K-Means (K-Médias): algoritmo que divide os dados em K grupos (clusters), onde cada país pertence ao grupo cujo **centro (centroide)** é o **mais próximo**.

O algoritmo funciona assim:

1. Escolhe K pontos aleatórios como centroides iniciais
2. Atribui cada país ao centroide mais próximo
3. Recalcula os centroides como a média dos países no grupo
4. Repete os passos 2 e 3 até estabilizar

Visualização simplificada com K=3:



No nosso projeto, K=5 e os 5 grupos resultantes são:

- **Hydro Leaders** — países com matriz majoritariamente hidrelétrica (ex: Brasil, Noruega)
- **Wind Champions** — países com grande participação eólica (ex: Dinamarca, Uruguai)
- **Solar Pioneers** — países com alta participação solar (ex: Chile, Itália)
- **Fossil Dependent** — países dependentes de carvão e gás (ex: maioria dos países em desenvolvimento)
- **Mixed Renewables** — países com mix equilibrado (ex: Suécia, Suíça)

■ BLOCO 1 — Constantes

```
N_CLUSTERS    = 5
RANDOM_STATE   = 42
N_INIT        = 10

FEATURES = [
    "renewables_share_elec",
    "solar_share_elec",
    "wind_share_elec",
    "hydro_share_elec",
    "fossil_share_elec",
]
```

`RANDOM_STATE = 42`

K-Means tem elementos aleatórios na inicialização dos centroides. `random_state=42` é uma **semente aleatória** — garante que ao executar o código duas vezes, você obterá o mesmo resultado. O número 42 é tradição (referência ao livro "O Guia do Mochileiro das Galáxias"), mas poderia ser qualquer inteiro.

`N_INIT = 10`

K-Means pode ficar preso em mínimos locais dependendo da inicialização. `n_init=10` faz o algoritmo rodar 10 vezes com inicializações diferentes e escolhe o melhor resultado (menor inércia). Isso aumenta a robustez.

`FEATURES`

Estas são as **variáveis (features)** que o algoritmo usará para agrupar os países. Pense nelas como as dimensões do espaço onde os países existem. Um país é um ponto em um espaço de 5 dimensões:

```
Brasil = (86.6, 11.8, 15.7, 51.8, 13.4)
          ↑   ↑   ↑   ↑   ↑
          renov solar wind hydro fóssil
```

■ BLOCO 2 — Função `run_clustering`

```
def run_clustering(
    df: pd.DataFrame,
    year: int = 2024,
    n_clusters: int = N_CLUSTERS,
) -> tuple[pd.DataFrame, dict, list[dict]]:
```

Múltiplos valores de retorno com `tuple`

Python permite retornar múltiplos valores de uma função usando tuplas:

```
return df_clust, cluster_profiles, scatter_data
# equivale a:
return (df_clust, cluster_profiles, scatter_data)
```

Quem chama a função pode desempacotar:

```
df_clust, profiles, scatter = run_clustering(df)
```

■ BLOCO 3 — Normalização com StandardScaler

```
scaler = StandardScaler()
X = scaler.fit_transform(df_clust[FEATURES])
```

Por que normalizar? — Conceito CRÍTICO em ML

Imagine que você quer medir a "distância" entre dois países considerando:

- Participação renovável: varia de 0% a 100%
- Participação solar: varia de 0% a 45%

Se não normalizar, a variável com maior escala **domina** o cálculo de distância. Por exemplo:

```
Brasil: renovável=86.6, solar=11.8 → ponto (86.6, 11.8)
China: renovável=37.0, solar=6.9 → ponto (37.0, 6.9)
Noruega: renovável=98.0, solar=0.1 → ponto (98.0, 0.1)

Distância Brasil→China:  $\sqrt{(86.6-37.0)^2 + (11.8-6.9)^2} \approx 49.6$ 
Distância Brasil→Noruega:  $\sqrt{(86.6-98.0)^2 + (11.8-0.1)^2} \approx 16.0$ 
```

Sem normalizar, a componente "renovável" (que vai a 100%) domina a de "solar" (que vai a 45%).

StandardScaler transforma cada feature para média 0 e desvio padrão 1:

```
z = (x - média) / desvio_padrao
```

Após transformação, todas as features têm a mesma escala e peso igual no K-Means.

`fit_transform` vs `fit` + `transform`

- `fit()` — aprende a média e o desvio padrão dos dados
- `transform()` — aplica a transformação
- `fit_transform()` — faz os dois em uma chamada (atalho conveniente)

■ BLOCO 4 — Aplicando o K-Means

```
km = KMeans(n_clusters=n_clusters, random_state=RANDOM_STATE, n_init=N_INIT)
df_clust = df_clust.copy()
df_clust["cluster_id"] = km.fit_predict(X)

log.info("Inertia: %.2f | Iterações: %d", km.inertia_, km.n_iter_)
```

`km.fit_predict(X)`

`fit_predict` é equivalente a `fit(X) + predict(X)` :

1. `fit(X)` — treina o modelo (encontra os K centroides)
2. `predict(X)` — para cada ponto, retorna o ID do cluster mais próximo (0, 1, 2, 3 ou 4)

O resultado é um array como:

```
[4, 0, 2, 4, 1, 0, 3, ...] # ID do cluster de cada país
```

`km.inertia_` — A Inércia

A inércia é a **soma das distâncias quadráticas** de cada ponto ao centroide do seu cluster. Quanto menor, mais compactos são os clusters (países mais similares dentro de cada grupo).

```
Inércia = Σ distance(ponto, centroide_do_cluster)2
```

`km.niter` — Número de iterações

Quantas vezes o algoritmo recalculou os centroides até convergir.

■ BLOCO 5 — Nomeação Automática dos Clusters (`nameclusters`)

```
def _name_clusters(df_clust: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    profiles = df_clust.groupby("cluster_id")[FEATURES].mean()

    assigned: dict[int, str] = {}

    hydro_c = int(profiles["hydro_share_elec"].idxmax())
    assigned[hydro_c] = "Hydro Leaders"

    wind_c = int(profiles[~profiles.index.isin(assigned)]["wind_share_elec"].idxmax())
    assigned[wind_c] = "Wind Champions"
    ...

    df_clust["cluster_name"] = df_clust["cluster_id"].map(assigned)
    return df_clust, assigned
```

`groupby().mean()` — Agregação

`df_clust.groupby("cluster_id")[FEATURES].mean()` é uma das operações mais poderosas do pandas. Ela:

1. Agrupa as linhas por `cluster_id` (0, 1, 2, 3, 4)
2. Para cada grupo, calcula a média de cada feature

Resultado (tabela):

cluster_id	renewables	solar	wind	hydro	fossil	
0	88.5	2.7	1.9	78.7	10.5	← Hydro Leaders
1	42.6	22.5	4.5	11.0	53.3	← Solar Pioneers
2	48.4	4.1	3.9	36.7	44.6	← Mixed Renewables
3	65.5	10.9	30.5	11.6	25.3	← Wind Champions
4	10.8	3.7	1.6	4.2	87.3	← Fossil Dependent

`idxmax()` — Índice do valor máximo

Retorna o **índice** (neste caso, o `cluster_id`) da linha com o maior valor na coluna. Assim podemos identificar automaticamente qual cluster tem maior hidro, maior vento, etc.

`~profiles.index.isin(assigned)` — Negação de máscara

- `profiles.index.isin(assigned)` → array `[True, False, True, ...]` — quais clusters já foram atribuídos
- `~` → inverte (NOT lógico) → seleciona apenas os **não atribuídos**

Isso evita que dois clusters recebam o mesmo nome.

`.map(assigned)` — Substituição em massa

Substitui cada `cluster_id` (número inteiro) pelo nome correspondente no dicionário `assigned` :

```
0 → "Hydro Leaders"
3 → "Wind Champions"
4 → "Fossil Dependent"
```

■ BLOCO 6 — Perfis dos Clusters (`buildprofiles`)

```
for name in cluster_order:
    grp = df_clust[df_clust["cluster_name"] == name]
    if grp.empty:
        continue
    top5 = (
        grp.sort_values("renewables_share_elec", ascending=False)["country"]
        .head(5)
        .tolist()
    )
```

`grp.empty` — Verificação de DataFrame vazio

Retorna `True` se o DataFrame não tiver nenhuma linha. `continue` pula para a próxima iteração do loop.

`.head(5).tolist()`

Encadeamento de métodos (method chaining) — cada método retorna um objeto, e você chama o próximo método sobre ele:

1. `.sort_values(...)` → ordena
2. `["country"]` → seleciona a coluna país
3. `.head(5)` → pega as 5 primeiras linhas
4. `.tolist()` → converte de pandas Series para lista Python

■ BLOCO 7 — Scatter Data (`buildscatter`)

```
def _build_scatter(df_clust: pd.DataFrame) -> list[dict]:
    return [
        {
            "country": r["country"],
            "renewables": round(float(r["renewables_share_elec"]), 1),
            "fossil": round(float(r["fossil_share_elec"]), 1),
            "cluster": r["cluster_name"],
            "continent": COUNTRY_TO_CONTINENT.get(r["country"], "Outros"),
        }
        for _, r in df_clust.iterrows()
    ]
```

List comprehension com `iterrows()`

Percorre cada linha do DataFrame e cria um dicionário por país. O resultado é uma lista de dicionários, perfeita para o gráfico de dispersão (scatter plot) no dashboard.

`.get(key, default)`

`COUNTRY_TO_CONTINENT.get(r["country"], "Outros")` tenta buscar o continente. Se não encontrar, retorna "Outros" em vez de lançar um erro `KeyError`.

ARQUIVO 4: `forecast.py`

Machine Learning com Regresso Polinomial

■ Conceito Fundamental: Regressão Polinomial

Regressão Linear Simples tenta ajustar uma **linha reta** aos dados:

$$y = \beta_0 + \beta_1 \cdot x$$

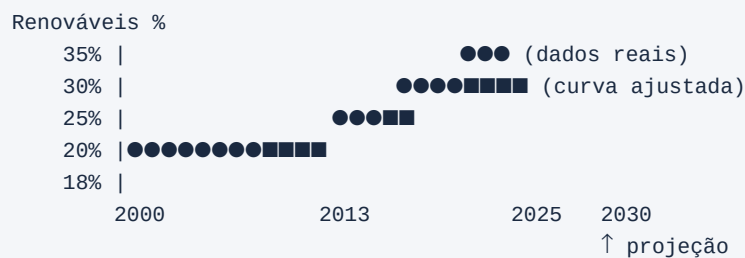
Mas nem sempre os dados seguem uma linha reta. A participação de renováveis, por exemplo, cresceu devagar até ~2012 e acelerou depois. Isso é uma **curva**, não uma linha.

Regressão Polinomial ajusta uma **curva** adicionando potências de x:

Grau 1 (linear): $y = \beta_0 + \beta_1 \cdot x$
 Grau 2 (quadrática): $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2$
 Grau 3 (cúbica): $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \beta_3 \cdot x^3$

No nosso projeto, **grau 2** é suficiente e obtém $R^2 = 0.996$ (ajuste quase perfeito!).

Visualização:



■ BLOCO 1 — Preparação dos Dados

```
series = world.sort_values("year").dropna(subset=[TARGET_COL])
years = series["year"].values.reshape(-1, 1)
y = series[TARGET_COL].values
```

`series["year"].values`

`.values` extrai os dados como um **array NumPy** (em vez de pandas Series). Arrays NumPy são necessários para o scikit-learn.

`.reshape(-1, 1)` — Reformatar o array

```
Antes: [2000, 2001, 2002, ..., 2025]    shape: (26,)
Depois: [[2000],
         [2001],
         [2002],
         ...,
         [2025]]
         shape: (26, 1) — 26 linhas, 1 coluna
```

O scikit-learn espera que X seja uma **matriz 2D** (linhas × colunas), mesmo que tenha só 1 feature. O `-1` no reshape significa "calcule automaticamente esta dimensão".

■ BLOCO 2 — Transformação Polinomial

```
poly = PolynomialFeatures(degree=degree, include_bias=True)
X_poly = poly.fit_transform(years)
```

`PolynomialFeatures` — A mágica da regressão polinomial

Este é um pré-processador que **cria novas colunas** a partir da coluna original:

Entrada (degree=2):	Saída:		
[[2000],	[1,	2000,	4000000],
[2001],	[1,	2001,	4004001],
[2002],	[1,	2002,	4008004],
...	...		
[2025]]	[1,	2025,	4100625]]
↑	↑	↑	↑
x	bias	x	x ²

`include_bias=True` adiciona a coluna de 1s (para o termo independente β_0).

Por que isso funciona?

Ao criar as colunas $[1, x, x^2]$, a regressão "linear" agora é na verdade polinomial:

$$\hat{y} = \beta_0 \cdot 1 + \beta_1 \cdot x + \beta_2 \cdot x^2$$

O scikit-learn trata cada coluna como uma feature independente e faz a regressão linear normalmente — mas o resultado é uma curva!

■ BLOCO 3 — Ajuste OLS (Mínimos Quadrados Ordinários)

```
model = LinearRegression(fit_intercept=False)
model.fit(X_poly, y)
```

`fit_intercept=False`

Como `PolynomialFeatures(include_bias=True)` já adicionou a coluna de 1s (que representa o intercepto), desligamos o intercepto interno do `LinearRegression` para evitar duplicidade.

```
model.fit(X_poly, y)
```

O método `fit` executa a estimação por **Mínimos Quadrados Ordinários (OLS)**. A matemática por trás:

Queremos encontrar os coeficientes β que **minimizem a soma dos erros quadráticos**:

$$\min \sum (y_i - \hat{y}_i)^2 = \min \sum (y_i - \beta_0 - \beta_1 \cdot x_i - \beta_2 \cdot x_i^2)^2$$

A solução analítica é:

$$\beta = (X^T X)^{-1} \cdot X^T y$$

O scikit-learn resolve isso eficientemente internamente.

■ BLOCO 4 — Métricas de Avaliação

```
y_pred = model.predict(X_poly)
r2      = model.score(X_poly, y)
mae     = mean_absolute_error(y, y_pred)
rmse    = float(np.sqrt(mean_squared_error(y, y_pred)))
```

R² — Coeficiente de Determinação

Mede o quanto o modelo explica a variabilidade dos dados:

```
R² = 1 - (SS_residual / SS_total)

SS_residual = Σ (y - ŷ)² ← erro do modelo
SS_total    = Σ (y - ȳ)² ← variação total dos dados
```

R ²	Interpretação
R ² = 1.0	Ajuste perfeito (todos os pontos na curva)
R ² = 0.99	Ajuste excelente
R ² = 0.8	Ajuste bom
R ² = 0.5	Ajuste fraco
R ² = 0.0	O modelo não explica nada

Nosso modelo obteve $R^2 = 0.9960$ — excelente!

MAE — Mean Absolute Error (Erro Absoluto Médio)

$$\text{MAE} = (1/n) \cdot \sum |y - \hat{y}|$$

A **média dos erros absolutos**. MAE = 0.24 pp significa que, em média, o modelo erra 0.24 pontos percentuais. Fácil de interpretar.

RMSE — Root Mean Squared Error (Raiz do Erro Quadrático Médio)

$$\text{RMSE} = \sqrt{(1/n) \cdot \sum (y - \hat{y})^2}$$

Similar ao MAE, mas **penaliza erros grandes** mais severamente (por causa do quadrado). RMSE = 0.31 pp.

```
np.sqrt(meansquarederror(...))
```

`meansquarederror` retorna o MSE (sem a raiz). Aplicamos `np.sqrt()` para obter o RMSE na mesma unidade dos dados (pontos percentuais).

■ BLOCO 5 — Projeção Futura

```
future_years = np.arange(FORECAST_YEAR_START, forecast_end + 1).reshape(-1, 1)
X_fut        = poly.transform(future_years)
y_fut        = model.predict(X_fut)
```

```
np.arange(2026, 2031)
```

Cria um array `[2026, 2027, 2028, 2029, 2030]`. Note que o limite superior é exclusivo (por isso `forecast_end + 1`).

`poly.transform()` (**sem fit**)

Aqui usamos `transform()` sozinho, sem `fit()`. Por quê? Porque o `poly` **já foi treinado** nos dados históricos (2000–2025). Agora estamos apenas aplicando a mesma transformação aos anos futuros — sem "aprender" nada novo.

Regra de ouro em ML: **nunca re-treine o pré-processador nos dados de teste/futuro.**

```
model.predict(X_fut)
```

Aplica os coeficientes aprendidos aos dados futuros transformados:

```

##### =  $\beta_0 + \beta_1 \cdot 2026 + \beta_2 \cdot 2026^2$ 
##### =  $\beta_0 + \beta_1 \cdot 2027 + \beta_2 \cdot 2027^2$ 
...

```

■ BLOCO 6 — Cálculo do Delta

```

prev_val = float(series[series["year"] == series["year"].max()][TARGET_COL].values[0])
forecast_list = []
for yr, val in zip(future_years.flatten(), y_fut):
    forecast_list.append({
        "year": int(yr),
        "value": round(float(val), 2),
        "delta": round(float(val) - prev_val, 2),
    })
    prev_val = float(val)

```

`zip()` — Iteração paralela

`zip(future_years.flatten(), y_fut)` emparelha os anos com os valores previstos:

```

zip([2026, 2027, 2028], [34.80, 36.25, 37.76])
→ [(2026, 34.80), (2027, 36.25), (2028, 37.76)]

```

`future_years.flatten()`

Converte a matriz de forma (5, 1) de volta para um array 1D (5,). Necessário para iterar com `zip`.

`prev_val = float(val)` dentro do loop

`prev_val` é atualizado a cada iteração para calcular o delta em relação ao **ano anterior** (não sempre em relação a 2025). Assim:

```

delta 2026 = 34.80 - 33.76 = +1.04 pp (em relação a 2025)
delta 2027 = 36.25 - 34.80 = +1.45 pp (em relação a 2026)
delta 2028 = 37.76 - 36.25 = +1.51 pp (em relação a 2027)

```

`model.coef_`

Após o treinamento, `model.coef_` contém os coeficientes estimados:

```
[β█, β█, β█] = [-999999.x, 998.x, -0.248]
```

(os valores exatos dependem dos dados)

ARQUIVO 5: `eda.py`

Anlise Exploratria de Dados

■ Conceito: O que é EDA?

EDA (Exploratory Data Analysis) é o primeiro passo em qualquer projeto de Ciência de Dados. Antes de construir modelos, você precisa **entender seus dados**:

- Quantas linhas e colunas existem?
- Há valores ausentes (NaN)?
- Quais são os valores típicos, mínimos e máximos?
- Existem outliers (valores extremos)?
- Como as variáveis se relacionam?

É como um médico que faz um check-up completo antes de prescrever um tratamento.

■ BLOCO 1 — Função `inspect_general`

```
def inspect_general(df: pd.DataFrame) -> None:
    for col in df.columns:
        miss = df[col].isna().sum()
        pct = miss / len(df) * 100
        print(f" {col:<40} dtype={str(df[col].dtype):<10} nulos={miss:>5} ({pct:5.1f}%)"
```

```
df[col].isna().sum()
```

- `isna()` → retorna `True` para cada célula que é `NaN`, `False` para as demais

- `.sum()` → soma os `True`s (Python trata `True` como 1)

Resultado: número de valores ausentes na coluna.

f-strings com formatação avançada

```
f" {col:<40} dtype={str(df[col].dtype):<10} nulos={miss:>5} ({pct:5.1f}%)"
```

Formato	Significado
<code>{col:<40}</code>	Coluna com 40 chars, alinhada à esquerda (<code><</code>)
<code>{dtype:<10}</code>	Tipo com 10 chars, alinhado à esquerda
<code>{miss:>5}</code>	Número com 5 chars, alinhado à direita (<code>></code>)
<code>{pct:5.1f}</code>	Float com 5 chars totais, 1 casa decimal

Isso garante que a saída fique em colunas alinhadas, facilitando a leitura.

■ BLOCO 2 — `inspect_stats`

```
stats = real[KEY_COLS].describe().round(2)
print(f"\n{stats.to_string()}")
```

`df.describe()` — Estatísticas descritivas em uma linha

Calcula automaticamente para cada coluna numérica:

```

renewables_share_elec
count          5549.00  ← total de valores não-nulos
mean            32.45   ← média
std             29.88   ← desvio padrão
min              0.00   ← mínimo
25%              9.44   ← 1º quartil (25% dos dados abaixo)
50%             22.97   ← mediana (50% dos dados abaixo)
75%             56.37   ← 3º quartil (75% dos dados abaixo)
max            100.00   ← máximo

```

Quartis e desvio padrão — Conceitos Estatísticos Básicos

- **Média:** soma dividida por N → sensível a outliers

- **Mediana (50%):** valor central → robusta a outliers
- **Desvio padrão:** "distância média" dos valores em relação à média → mede dispersão
- **Quartis:** dividem os dados em 4 partes iguais de 25% cada

■ BLOCO 3 — inspect_correlations

```
corr = df[mask][KEY_COLS].corr().round(3)
```

`.corr()` — Matriz de Correlação de Pearson

Calcula o **coeficiente de correlação** entre cada par de variáveis, variando de -1 a +1:

Valor	Interpretação
+1.0	Correlação positiva perfeita
+0.7	Correlação positiva forte
0.0	Sem correlação
-0.7	Correlação negativa forte
-1.0	Correlação negativa perfeita

Exemplo intuitivo:

```

renewables  fossil
renewables_share    1.000    -0.913
fossil_share       -0.913     1.000

```

Faz sentido! Países com alta participação renovável têm baixa participação fóssil (correlação negativa forte de -0.913).

Esta análise é valiosa porque features altamente correlacionadas podem causar problemas em alguns modelos de ML (multicolinearidade).

Resumo dos Conceitos Aprendidos

Python Puro

Conceito	Onde vimos
<code>import</code>	Todos os arquivos
Funções (<code>def</code>) e docstrings	Todos os arquivos
Type hints (<code>: Type</code> , <code>-&gt; Type</code>)	Todos os arquivos
<code>if __name__ == "__main__"</code>	Todos os arquivos
Dict / List comprehensions	<code>process_data.py</code> , <code>clustering.py</code>
<code>with open()</code>	<code>process_data.py</code> , <code>clustering.py</code> , <code>forecast.py</code>
<code>pathlib.Path</code>	<code>process_data.py</code> , <code>forecast.py</code>
<code>argparse</code>	<code>process_data.py</code> , <code>clustering.py</code> , <code>forecast.py</code>
<code>logging</code>	Todos os arquivos
Funções aninhadas (nested)	<code>process_data.py</code>
Múltiplos retornos	<code>clustering.py</code>

Pandas

Operação	Onde vimos
<code>pd.read_csv()</code>	<code>process_data.py</code>
Seleção por coluna <code>df["col"]</code>	Todos
Máscara booleana <code>df[mask]</code>	<code>process_data.py</code> , <code>clustering.py</code>
<code>.copy()</code>	<code>process_data.py</code> , <code>clustering.py</code>
<code>.dropna()</code>	<code>forecast.py</code>
<code>.fillna()</code>	<code>process_data.py</code>
<code>.map()</code>	<code>process_data.py</code> , <code>clustering.py</code>
<code>.merge()</code> (JOIN)	<code>process_data.py</code>
<code>.groupby().mean()</code>	<code>clustering.py</code>
<code>.sort_values()</code>	Todos
<code>.reset_index()</code>	<code>process_data.py</code>

Operação	Onde vimos
<code>.diff()</code>	<code>process_data.py</code>
<code>.describe()</code>	<code>eda.py</code>
<code>.corr()</code>	<code>eda.py</code>
<code>.iterrows()</code>	<code>process_data.py</code> , <code>clustering.py</code>

NumPy

Operação	Onde vimos
<code>.values</code>	<code>forecast.py</code>
<code>.reshape(-1, 1)</code>	<code>forecast.py</code>
<code>np.arange()</code>	<code>forecast.py</code>
<code>.flatten()</code>	<code>forecast.py</code>
<code>np.sqrt()</code>	<code>forecast.py</code>

Scikit-learn (Machine Learning)

Conceito	Onde vimos
<code>StandardScaler</code> — normalização	<code>clustering.py</code>
<code>KMeans</code> — agrupamento	<code>clustering.py</code>
<code>km.fit_predict()</code>	<code>clustering.py</code>
<code>km.inertia_</code>	<code>clustering.py</code>
<code>PolynomialFeatures</code> — features polinomiais	<code>forecast.py</code>
<code>LinearRegression</code> — regressão OLS	<code>forecast.py</code>
<code>model.fit()</code>	<code>forecast.py</code>
<code>model.predict()</code>	<code>forecast.py</code>
<code>model.score()</code> — R^2	<code>forecast.py</code>
<code>meanabsoluteerror</code> — MAE	<code>forecast.py</code>
<code>meansquarederror</code> — MSE/RMSE	<code>forecast.py</code>

Prximos Passos para o Estudante

Experimentos que você pode fazer:

1. **Mude o K do K-Means:** Execute `python -m scripts.clustering --k 3` e compare os clusters. O que muda?
2. **Mude o grau da regressão:** Execute `python -m scripts.forecast --degree 3`. O R^2 melhora? A projeção muda muito?
3. **Mude o horizonte:** Execute `python -m scripts.forecast --horizon 2040`. A curva ainda faz sentido ou começa a extrapolar demais?
4. **Explore com EDA:** Execute `python -m scripts.eda` e leia toda a saída. Que países te surpreenderam no ranking?
5. **Visualize o clustering:** No Jupyter Notebook, plote o scatter plot dos clusters com matplotlib para visualizar a separação.

***In God we trust, all others must bring data.**

— W. Edwards Deming

Energix Global · Desenvolvido por Renan Pinheiro · 2026